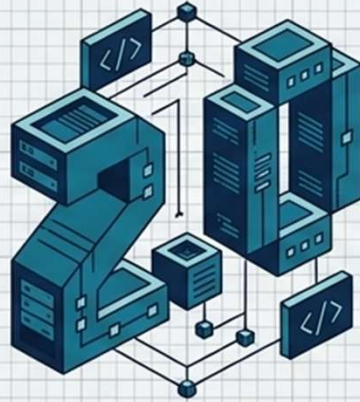


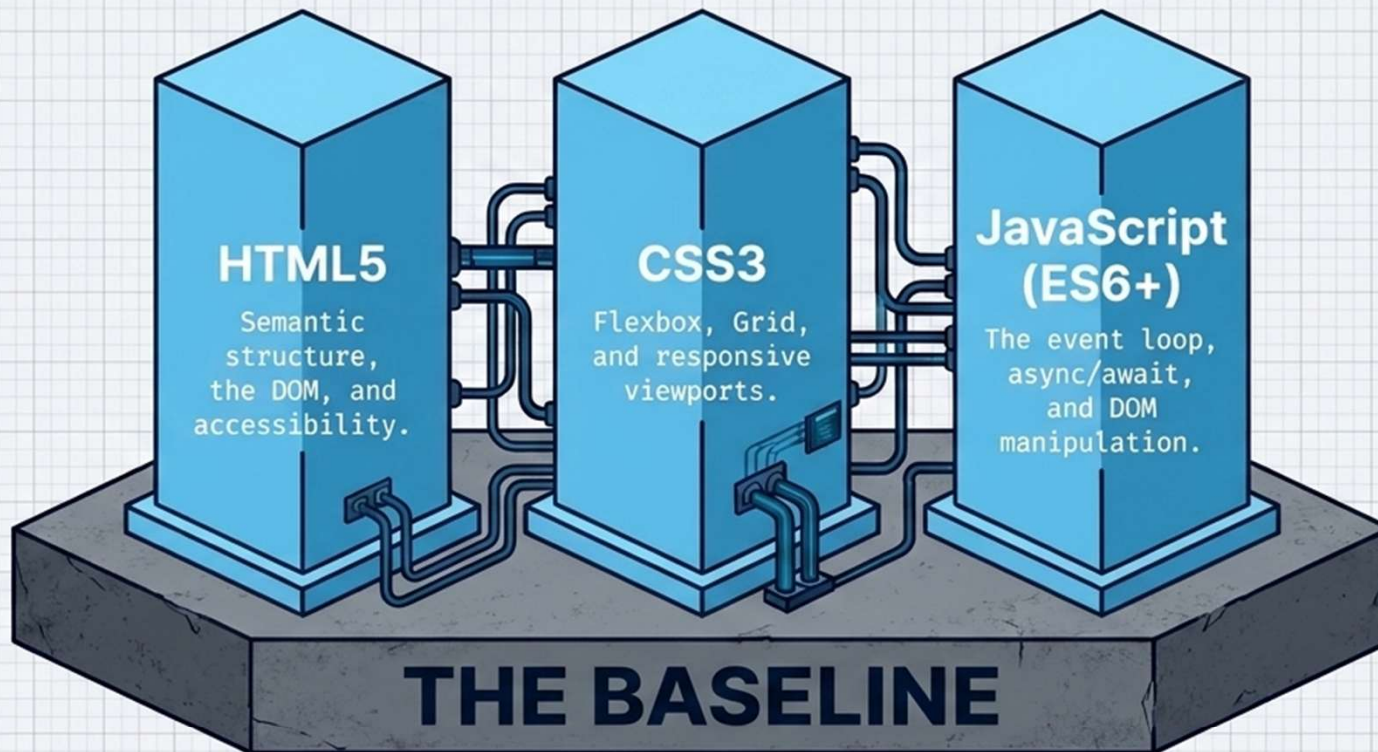


DEV-STUDIO

Building Developers for the AI Era

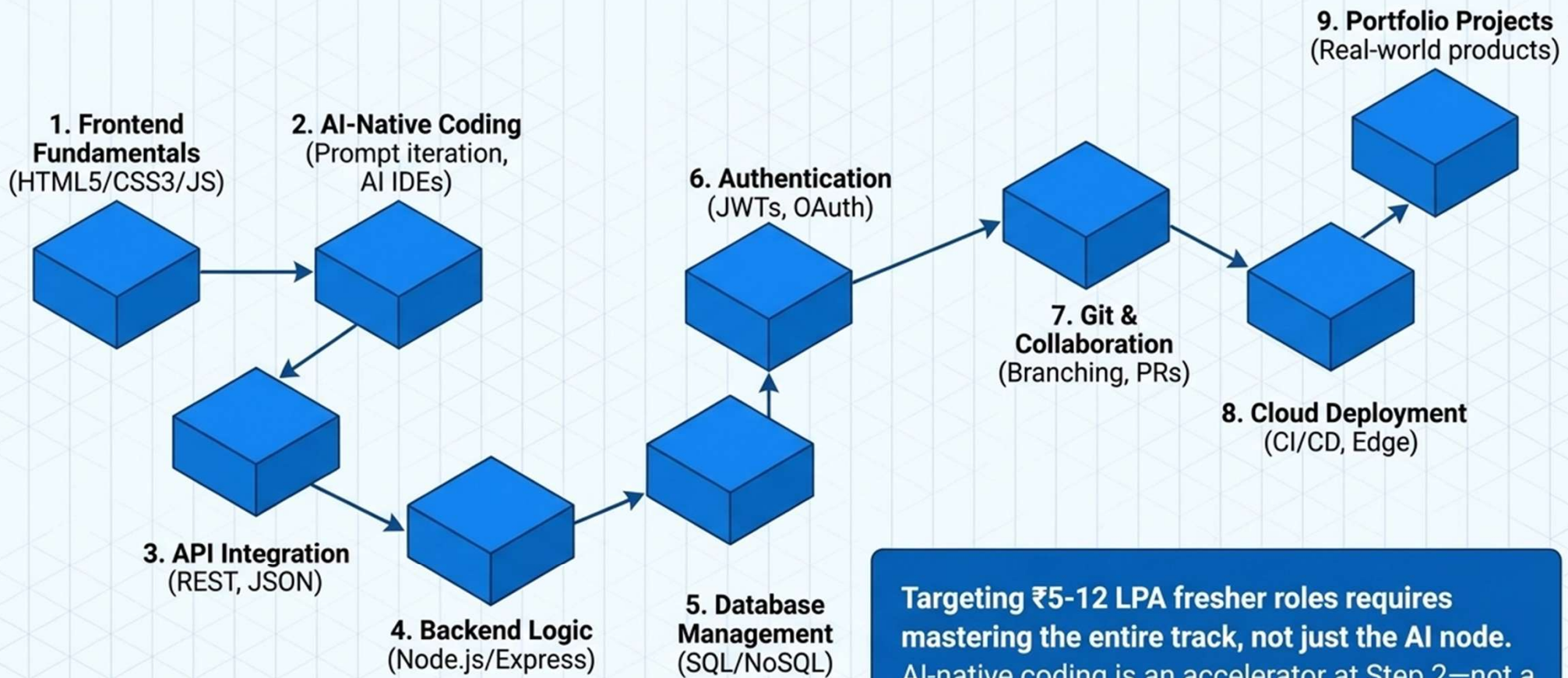
Prompting gets you a prototype; architectural literacy builds production software.

The Baseline: Fundamentals are Non-Negotiable



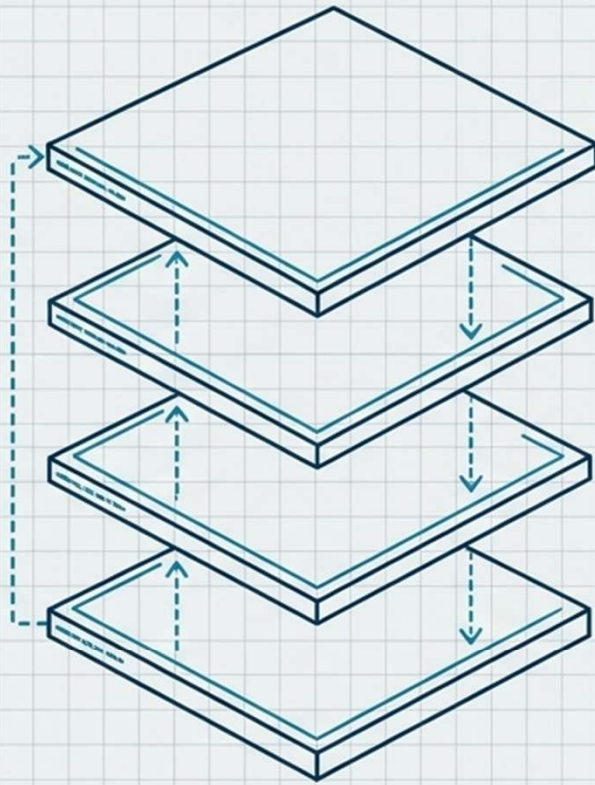
These are not optional. AI code generators output standard web primitives. You cannot audit or refactor generated code if you don't understand the raw syntax it compiles down to.

9-Step Roadmap to AI-Native Engineering



Targeting ₹5-12 LPA fresher roles requires mastering the entire track, not just the AI node. AI-native coding is an accelerator at Step 2—not a shortcut that lets you skip Steps 3 through 9.

Full Stack Architecture: End-to-End Data Flow



Frontend (React/Next.js) - The UI and user interaction.

API (The Bridge) - REST endpoints transferring JSON.

Backend (Node.js/Spring Boot) - Business logic, validation, and security.

Database (PostgreSQL/MongoDB) - Persistent storage and memory.

When a Lovable or Bolt.new prototype breaks, elite engineers know exactly which layer is failing. Understanding data flow is your diagnostic tool.

Redesigned Slide C

AI IDEs & Assistants	AI App Builders	AI Design → Code	Backend & DB	Auth	Deployment	Model APIs
Cursor	Bolt.new	Google Stitch	Supabase	Clerk	Vercel	Claude
Google Antigravity	Lovable	Figma (Dev Mode)	Firebase	Auth.js	Netlify	OpenAI
GitHub Copilot	v0	Framer	Neon	Auth0	Render	Gemini
Claude Code	Replit Agent		PlanetScale		Railway	
Windsurf						



Tools change every quarter. Fundamentals don't. Learn to evaluate new tools, not just memorize this list. Mastering foundational protocols outlasts any single tool.

DEV-STUDIO Operations



WORKSHOPS

Deep dives into specific full-stack layers, architecture, and prompt auditing.



HACKATHONS

24-hour sprints to build, deploy, and solve real campus problems.



PROJECT SPRINTS

Multi-week collaborative builds mirroring real agile industry teams to transform concepts into deployed apps.



CODE REVIEWS & AUDITS

Peer-reviewing AI-generated code to catch architectural flaws and security risks.

We don't just watch tutorials—we build, review, and ship together.

DEV-STUDIO On Campus

Web development sessions, hands-on labs, and the crew behind it.



HANDS-ON LAB SESSION



THE TEAM



WEB DEVELOPMENT SESSION

Case Study: Campus Maintenance Portal

A real problem, mapped onto a full-stack build

Students report problems around campus

FACILITIES

"Projector in Room 302 isn't working."

HOSTEL

"Hostel Block B water cooler isn't working."

IT LAB

"Lab PC #17 isn't booting."

Each complaint routes straight to the concerned department.

The complaint lifecycle



WHY IT'S
A GREAT BUILD

Exercises **CRUD operations**, **authentication**, **role-based access** (student / department / technician), and **status management** end to end.

Case Study: Campus Bites

Skip the queue — order ahead from any campus food stall

The food court problem

QUEUES

Long lines at every stall during lunch and break — students lose their whole break waiting.

NO VISIBILITY

No way to know if an order is received, being prepared, or ready for pickup.

STALL OVERLOAD

Staff juggle walk-ins and a growing backlog with no system to manage orders.

Ordering ahead removes the queue entirely.

The order lifecycle



WHY IT'S A GREAT BUILD

Exercises **real-time status updates**, **role-based dashboards** (student / stall staff), **order queue management**, and **payment integration**.